MODULE 5.1 · Concept 4 of 7

Task Decomposition

Breaking complex goals into manageable sub-tasks — recursive, parallel, and sequential

8 min	B3 – Apply	L2 – Intermediate	TH-000007
Duration	Bloom's	Level	Prerequisite

AI-AG-IN-TH-000008

Prof. Sashikumaar Ganesan · IISc Bangalore · AhamX Bodhi

SECTION 1 — WHAT

Breaking Tasks Apart

Complex goal → manageable sub-tasks, each solvable by a single agent step

What Task Decomposition Is

Turning one complex goal into multiple simpler sub-tasks

Task decomposition is the process of breaking a complex goal into smaller, independent sub-tasks that are each simple enough for one agent step (one tool call, one reasoning cycle) to handle.

Without decomposition:

"Write a market research report on Indian IT sector"
→ Agent tries to do everything in one step
→ Overwhelmed, produces shallow or incomplete result
→ No clear progress tracking

With decomposition:

1. Search for market size data
2. Search for top companies + revenue
3. Search for growth trends
4. Analyze competitive landscape
5. Synthesize into report structure
6. Write each section → Compile final report

Why Decomposition Matters

Three benefits: reduced complexity, parallelism, progress visibility

Reduced Per-Step Complexity

Each sub-task is simple enough for one tool call or one reasoning step. The agent doesn't need to hold the entire problem in mind.

Instead of 'research the market', just 'search for market size data' — one search query.

Parallel Execution

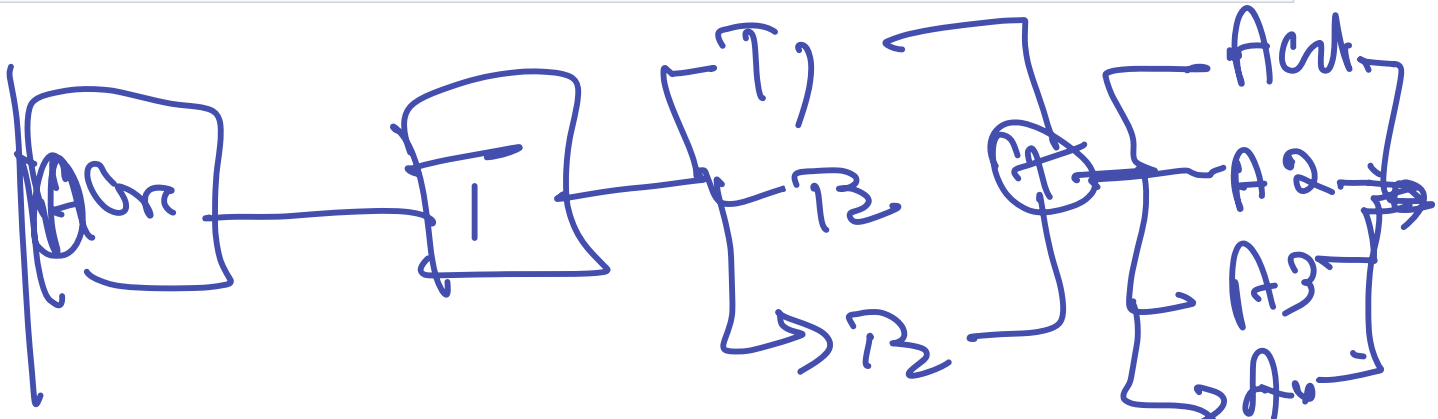
Independent sub-tasks can run simultaneously. Steps 1, 2, and 3 in the report example don't depend on each other.

3 parallel searches complete in 2 seconds instead of 6 seconds sequentially.

Progress Visibility

Each completed sub-task is a measurable milestone. The user (or system) can see: '4/6 steps done, 67% complete'.

If step 3 fails, you know exactly which part failed — not just 'report generation failed'.



Recursive Decomposition: Tree Structure

Break task → break sub-tasks → break sub-sub-tasks (until each is atomic)

Each leaf node is an atomic task: one search query, one calculation, one writing step.
Recursion stops when a sub-task can be handled by a single tool call.

Parallel vs. Sequential Decomposition

Which sub-tasks can run simultaneously?

Parallel (Independent)

Sub-tasks that DON'T need each other's output:

- Search market size ← independent
- Search top companies ← independent
- Search growth trends ← independent

All three can run at the same time.
Total time: max(individual times)
not sum(individual times).

3 searches × 2 sec each = 2 sec total (not 6).

Sequential (Dependent)

Sub-tasks that NEED previous results:

- Gather data → then analyze it
- Analyze → then write report
- Write draft → then review/edit

Must run in order.
Each step needs output from previous.

Dependency chain determines critical path.

Dependency Management: The DAG Structure

Which sub-task depends on which? — Directed Acyclic Graph

Goal: 'Write market research report on Indian IT sector'

Sub-task	Depends On	Can Parallelize With	Output
1. Search market size	—	Tasks 2, 3	Market size data
2. Search top companies	—	Tasks 1, 3	Company list + revenue
3. Search growth trends	—	Tasks 1, 2	Trend data
4. Analyze landscape	Tasks 1, 2, 3	—	Analysis summary
5. Write report sections	Task 4	—	Draft sections
6. Compile final report	Task 5	—	Final PDF

Critical path: Tasks 1–3 (parallel, 2 sec) → Task 4 (3 sec) → Task 5 (5 sec) → Task 6 (2 sec) = 12 sec total.

Decomposition Depth: The Sweet Spot

Too shallow fails to simplify. Too deep adds coordination overhead.

Depth	Tasks	Per-Task Complexity	Coordination Overhead	Risk
Too Shallow	2–3	High (each task is still complex)	Low	Agent overwhelmed per step
Sweet Spot	5–8	Medium (each = 1–2 tool calls)	Medium	Best balance
Too Deep	15–20+	Very low (trivial tasks)	High	More time coordinating than doing

Rule of thumb: Decompose until each sub-task can be described in one sentence AND handled by one tool call. If you need more than one sentence to describe a sub-task, decompose further. If the sub-task is trivially simple (like 'add 2+3'), you've gone too deep.

SECTION 2 — HOW

Practical Decomposition

LLM prompts, quality evaluation, and failure modes

Worked Example: 'Market Research Report' Decomposed

Complete decomposition with dependencies, tools, and expected outputs

Goal: "Write a market research report on the Indian IT sector"

Decomposition (LLM-generated):

```
1. [PARALLEL] Search "Indian IT market size 2025"           → web_search  → ₹X trillion
2. [PARALLEL] Search "top 10 Indian IT companies revenue"     → web_search  → company list
3. [PARALLEL] Search "Indian IT sector growth trends 2024-25"  → web_search  → trend data
4. [SEQ after 1-3] Analyze: compare companies, identify trends → LLM reason  → analysis
5. [SEQ after 4] Write report sections: intro, market, firms   → LLM generate → draft
6. [SEQ after 5] Format as PDF with charts and citations       → doc_writer  → final.pdf
```

Parallel steps: 1,2,3 (2 sec) | Sequential: 4→5→6 (10 sec) | Total: ~12 sec

LLM Decomposition Prompts

How to get structured task breakdowns from LLMs

```
System prompt for decomposition:

"Break the user's task into numbered sub-tasks.
For each sub-task, output JSON:
{
  "step": 1,
  "description": "Search for Indian IT market size data",
  "tool": "web_search",
  "depends_on": [],
  "expected_output": "Market size in USD/INR with source",
  "can_parallel": true
}
List ALL sub-tasks needed. Each sub-task should be completable
with ONE tool call. If a sub-task needs multiple calls, decompose further."
```

The key is specifying the output format + the atomicity rule ('one tool call per sub-task').

Evaluating Decomposition Quality + Failure Modes

How to tell if the decomposition is good — and common mistakes

Quality Check	Good Decomposition	Bad Decomposition
Completeness	All aspects of goal covered	Missing sub-tasks (no data gathering step)
Correct ordering	Dependencies respected	Step 4 needs Step 5's output
Appropriate granularity	Each step = 1 tool call	Step is still a complex multi-step task
No redundancy	Each step is unique	Two steps search for the same data
Parallelism identified	Independent steps marked	Everything listed as sequential

Common failure: over-decomposition — 20 sub-tasks for a 3-step problem. Each coordination point adds latency and error risk.

Key Takeaways

The essential points from this concept

- **Decomposition = breaking complex goals into atomic sub-tasks** (one tool call each).
- **Three types: recursive (tree), parallel (independent), sequential (dependent).**
- **DAG structure tracks dependencies** — determines what can parallelize and the critical path.
- **Sweet spot: 5–8 sub-tasks.** Too shallow fails to simplify; too deep adds overhead.
- **LLM decomposition prompts work well** when you specify output format + atomicity rule.

Next: AI-AG-IN-TH-000009 — Goal-Directed Behavior

AI-AG-IN-TH-000008 Complete

Task Decomposition

Concept 4 of 8

Advanced Certification in Agentic & Generative AI · IISc Bangalore

Prof. Sashikumaar Ganesan · AhamX Bodhi